Vietnamese German University

Department of Computer Science and Engineering

COURSE PROJECT REPORT

Course Title: [Real-Time Systems / RTOS]

Smart Controller System using RTOS

Group Members

1	10423053	Nguyễn Hà Khải
2	10423056	Nguyễn Duy Khánh
3	10423093	Tô Huỳnh Phúc
4	10423097	Ninh Hoàng Quân
5	10423028	Võ Minh Duy

Project Requirements (Reference)

Objective: Develop an RTOS-based Smart Climate Control System that monitors temperature and humidity, and controls a heater, cooler, and humidifier (represented by LEDs) to maintain target conditions. The project is a course demo and need not reflect real-world physical operation.

Hardware: YoluUNO (ESP32-S3) microcontroller; DHT20 temperature/humidity sensor on I2C; three two-color LED actuators representing Heater (D3/D4), Cooler (D5/D6), and Humidifier (D7/D8); onboard status LED on GPIO48.

RTOS: FreeRTOS (or any RTOS suitable for the microcontroller), using the task scheduler.

Required behaviors: Serial print of temperature/humidity every 5 seconds; onboard LED blinks every 1 second; Heater LED shows GREEN/ORANGE/RED according to 3 defined temperature thresholds; Cooler activates for a fixed duration (e.g., 5 s) when temperature exceeds a threshold, then re-checks; Humidifier runs a GREEN (5 s) $\rightarrow$ YELLOW (3 s) $\rightarrow$ RED (2 s) sequence when humidity falls below a threshold, then re-checks humidity to decide whether to repeat.

Required tasks: Blinky Task, Read Temperature Task, Cooler Task, Heater Task, Humidifier Task.

Deliverable: GitHub repository link and result snapshots.

1. Introduction

1.1 Problem Description

The proposed system is an RTOS-based smart climate controller that continuously monitors environmental temperature and humidity using a DHT20 sensor. Based on the measured values, the system automatically controls visual actuator indicators represented by LEDs for the heater, cooler, and humidifier. The purpose of the system is to maintain environmental conditions within predefined ranges and demonstrate real-time task scheduling and coordination on an ESP32-S3 microcontroller.

Temperature and humidity regulation is important in many applications such as agriculture, storage facilities, and greenhouse management. Maintaining suitable environmental conditions can improve productivity and reduce the risk of damage caused by excessive heat, low temperature, or insufficient humidity. In this project, the physical actuators are represented by LEDs; however, the same control logic could be extended to real devices such as fans, heaters, and humidifiers.

1.2 Required Features

The system implements the following required monitoring and control functions:

- The DHT20 sensor measures temperature and humidity periodically every 5 seconds.
- The measured values are displayed through the serial monitor.
- The Heater indicator (LEDs D3 and D4) changes color according to the temperature level:
  - GREEN: temperature within the safe operating range.
  - ORANGE: temperature approaching a warning threshold.
  - RED: temperature in a critical range.
- The Cooler indicator (LEDs D5 and D6) is activated for a fixed duration of 5 seconds when the temperature exceeds the cooling threshold. After the activation period, the temperature is measured again to determine whether additional cooling is required.
- The Humidifier indicator (LEDs D7 and D8) executes a timed sequence when the humidity falls below the specified threshold:
  - GREEN for 5 seconds
  - YELLOW for 3 seconds
  - RED for 2 seconds
- After the sequence completes, the humidity value is checked again to determine whether the sequence should be repeated.
- The onboard status LED on GPIO48 blinks every second to indicate that the RTOS scheduler is running correctly.

1.3 Additional Proposed Features

In addition to the required functionality, an intermediate Decision Maker task is introduced between the sensor-monitoring task and the actuator tasks. The Decision Maker receives the latest temperature and

humidity measurements and determines which actuator tasks should be activated.

This additional component improves synchronization between tasks and prevents unintended repeated activations. According to the project requirements, sensor monitoring should resume only after the cooler or humidifier has completed its current operation. If the monitoring task directly signaled actuator tasks, a situation could occur in which one actuator finishes earlier than another and immediately triggers a new measurement cycle. For example, if the cooler finishes while the humidifier sequence is still running, the newly measured humidity value could cause the humidifier task to be activated again before its previous sequence has completed. Such behavior would lead to overlapping operations and potentially incorrect control decisions.

By introducing the Decision Maker, actuator execution is coordinated more effectively, ensuring that each control sequence completes before a new activation request is issued. This design demonstrates the use of RTOS synchronization concepts and improves the reliability of the overall control system.

2. System Design using State Machine

2.1 Overall System State Machine

The global system is modeled using an asynchronous, event-driven architecture utilizing FreeRTOS synchronization primitives (semaphores) to achieve non-blocking concurrency.

- **Continuous Monitoring Thread:** Operates as a high-priority periodic task. It updates ambient metrics continuously, flashing the status via the HEATER safety profile and refreshing the local LCD. Upon capturing stable data, it dispatches an activation token via Release `sensor_ready_sem()` to wake the decision-making engine.
- **Dynamic Fork-Join Concurrency Control:** The DECISION MAKER parses the sensor data and evaluates constraints. To execute adjustments concurrently without freezing peripheral systems, control flow forks into three prospective branches depending on active variables: thermal adjustment (COOLER), moisture adjustment (HUMIDIFIER), or an active bypass (Metrics are within the threshold).
- **Adaptive Synchronization Barrier:** Once paths are dispatched, the system blocks at a Join synchronization point. The feedback path safely routes control back to the baseline evaluation loop by implementing a dynamic conditional check: Acquire necessary semaphores if any. This guarantees that if an actuator runs, the decision-maker blocks until its fixed operational window finishes, while simultaneously ensuring that an idle loop instantly recycles without stalling.

2.2 Heater Control State Machine

This state machine manages the system safety levels. It controls the indicator LEDs (D3 and D4) using five distinct temperature boundaries as shown in the figure.

States Description

- **SAFE GREEN**: Default initial state on power-up, active during optimal ambient temperature conditions.
- **WARNING ORANGE**: Triggered during moderate thermal deviations, indicates active climate regulation is required.
- **CRITICAL RED**: Triggered during extreme thermal conditions, indicates high-risk environments requiring immediate intervention.

Transition Conditions

- **Initialization**: System boots directly into **SAFE GREEN**.
- **From SAFE GREEN**: Remains in state on a, moves to **WARNING ORANGE** on b OR c, moves to **CRITICAL RED** on d OR e.
- **From WARNING ORANGE**: Remains in state on b or c, moves to **CRITICAL RED** on d OR e, moves to **SAFE GREEN** on a.
- **From CRITICAL RED**: Remains in state on d or e, moves to **WARNING ORANGE** on b OR c, moves to **SAFE GREEN** on a.

2.3 Cooler Control State Machine

States Description

- **IDLE BLACK**: Default initial state when the system boots up, the cooler hardware remains completely powered **OFF**.
- **COOLING GREEN**: Active state indicating the physical cooler is running, the cooler runs for a strict, non-blocking time window.

Thresholds and Transition Conditions

- **Initialization**: The state machine directly enters **IDLE BLACK** upon system startup.
- **Activation Threshold**: The system transitions from **IDLE BLACK** to **COOLING GREEN** if the evaluated condition `[Temp > 28°C]` is met. This immediately triggers the physical cooler hardware to turn **ON**.
- **Fixed Duration**: Upon entering the **COOLING GREEN** state, a dedicated hardware or software timer begins counting. The cooler stays continuously active for exactly **5 seconds**.
- **Re-check Condition**: Once the condition `[Timer == 5s]` evaluates to true, the state machine turns the cooler **OFF** and transitions straight back to **IDLE BLACK**. On the very next clock cycle in the IDLE state, the temperature is re-checked against the 28°C activation threshold. If it is still hot, it cycles back into cooling; otherwise, it remains safely idle.

2.4 Humidifier Control State Machine

States Description

- **IDLE BLACK**: Default initial state upon hardware system power-up, the physical humidifier hardware remains powered **OFF**.
- **GREEN**: The initial phase of active humidification running for an exact, timed duration.
- **YELLOW**: The secondary intermediate phase of the active execution cycle.
- **RED**: The final phase of the humidification sequence before monitoring checks reset.

Activation Threshold and Transition Conditions

- **Initialization**: The state machine directly initializes into the **IDLE BLACK** state on startup.
- **Activation Threshold**: The system transitions from **IDLE BLACK** to **GREEN** when the condition `[Humi < 40]` evaluates to true. This transition instantly executes the action **Turn ON Humidifier**.
- **Phase 1 to Phase 2 (GREEN to YELLOW)**: The system remains in the GREEN phase for a fixed duration. Once the internal tracking variable meets the condition `[Time == 5s]`, it transitions to **YELLOW**.
- **Phase 2 to Phase 3 (YELLOW to RED)**: The system holds in the YELLOW state for an intermediate window. Once the condition `[Time == 3s]` evaluates to true, it transitions straight to **RED**.
- **Cycle Completion and Re-check (RED to IDLE BLACK)**: The system processes its final active interval in the RED state. When the condition `[Time == 2s]` is reached, the state machine drops

back into **IDLE BLACK**.

In the immediate next execution clock cycle inside **IDLE BLACK**, the relative humidity is re-evaluated. If `Humi < 40` remains true, the multi-phase cycle repeats immediately starting at **GREEN**; otherwise, the hardware drops out of operation and settles in the idle baseline.

3. System Implementation

3.1 Semaphore and Synchronization Strategy

The system uses a producer-consumer architecture with multiple semaphores and queues to synchronize concurrent RTOS tasks.

The Sensor Monitoring Task acts as the producer. Every 5 seconds, it reads the latest temperature and humidity values from the DHT20 sensor and stores them in a shared queue. After new data is available, the task releases the semaphore `sensor_ready_sem` to notify the Decision Maker Task.

The Decision Maker Task acts as the central coordinator. It waits for `sensor_ready_sem`, retrieves the newest sensor reading, and distributes the data to the LCD and Heater tasks through dedicated queues. Depending on the measured temperature and humidity, it may also activate the Cooler Task or Humidifier Task by releasing the corresponding semaphores.

Semaphore / Queue	Purpose
`sensor_ready_sem`	Signals that a new sensor sample is available.
`lcd_sem`	Signals that LCD data are ready to display.
`heater_sem`	Activates the heater task.
`cooler_sem`	Activates the cooler task.
`humidifier_sem`	Activates the humidifier task.
`cooler_finished_sem`	Signals that the cooler has completed its cycle.
`humidifier_finished_sem`	Signals that the humidifier has completed its sequence.

This design prevents race conditions between tasks and ensures that control actions are executed only when valid sensor data is available. Furthermore, the completion semaphores guarantee that the Decision Maker waits for actuator operations to finish before continuing with further control decisions.

3.2 Read Temperature Task

Priority: Medium

Period: 5 seconds

The Read Temperature Task periodically acquires temperature and humidity measurements from the DHT20 sensor. The collected data is stored in a shared queue and signaled through the `sensor_ready_sem` semaphore. This task acts as the producer in the producer-consumer communication pattern.

Main implementation:

```
async def task_monitoring():
    while True:
        data = {
            "temp": await dht20.atemperature(),
            "humi": await dht20.ahumidity()
        }
```

```
        add_latest(sensor_queue, sensor_ready_sem, data)
        await asleep_ms(5000)
```

The task continuously generates environmental measurements and provides them to the Decision Maker Task for further processing.

3.3 Cooler Task

Priority: High

Trigger Condition: Temperature $>$ 28°C

The Cooler Task is event-driven and remains blocked until the `cooler_sem` semaphore is released by the Decision Maker Task. Once activated, the cooler LED turns GREEN for a fixed duration of 5 seconds, representing the cooling process. After the timer expires, the LED is turned OFF and a completion signal is sent through `cooler_finished_sem`.

Main implementation:

```
async def task_cooler():
    while True:
        await cooler_sem.acquire()
        rgb_led_D5.show(0, hex_to_rgb('#00ff00'))
        await asleep_ms(5000)
        rgb_led_D5.show(0, hex_to_rgb('#000000'))
        cooler_finished_sem.release()
```

State Machine:

- IDLE (BLACK): Cooler inactive.
- COOLING (GREEN): Cooler active for 5 seconds.
- Return to IDLE after timer expiration.

The Cooler Task state machine is shown in Figure **??**.

3.4 Heater Task

Priority: Medium

Trigger Condition: New temperature reading available.

The Heater Task provides a visual indication of environmental temperature conditions. The task waits for a signal from `heater_sem`, retrieves the latest temperature value, and updates the RGB LED according to predefined temperature ranges.

Main implementation:

```
async def task_heater():
    while True:
        await heater_sem.acquire()
        temp = heater_queue.pop(0)["temp"]
        if 20 <= temp <= 25:
            rgb_led_D3.show(0, hex_to_rgb('#00ff00'))
        elif 15 <= temp <= 30:
            rgb_led_D3.show(0, hex_to_rgb('#ffff00'))
```

```
        else:
            rgb_led_D3.show(0, hex_to_rgb('#ff0000'))
```

State Machine:

- SAFE (GREEN): 20°C $\leq$ Temp $\leq$ 25°C
- WARNING (YELLOW): 15°C $\leq$ Temp $<$ 20°C or 25°C $<$ Temp $\leq$ 30°C
- CRITICAL (RED): Temp $<$ 15°C or Temp $>$ 30°C

The Heater Task continuously reflects the current thermal condition of the environment.

3.5 Humidifier Task

Priority: High

Trigger Condition: Humidity $<$ 40%

The Humidifier Task is activated when the humidity falls below the predefined threshold. After receiving a signal through `humidifier_sem`, it executes a timed three-stage sequence representing the humidification process.

Main implementation:

```
async def task_humidifier():
    while True:
        await humidifier_sem.acquire()
        rgb_led_D7.show(0, hex_to_rgb('#00ff00'))
        await asleep_ms(5000)
        rgb_led_D7.show(0, hex_to_rgb('#ffff00'))
        await asleep_ms(3000)
        rgb_led_D7.show(0, hex_to_rgb('#ff0000'))
        await asleep_ms(2000)
        rgb_led_D7.show(0, hex_to_rgb('#000000'))
        humidifier_finished_sem.release()
```

State Machine:

- IDLE (BLACK): Humidifier inactive.
- GREEN: Active humidification for 5 seconds.
- YELLOW: Intermediate phase for 3 seconds.
- RED: Final phase for 2 seconds.
- Return to IDLE and re-check humidity.

The completion semaphore informs the Decision Maker that the humidification cycle has finished.

4. Validation Results in Simulation

4.1 Monitoring Output

Figure **??** shows the LCD displaying the latest temperature and humidity values obtained from the DHT20 sensor. The monitoring task updates the measurements every 5 seconds and forwards the data to the Decision Maker Task for evaluation.

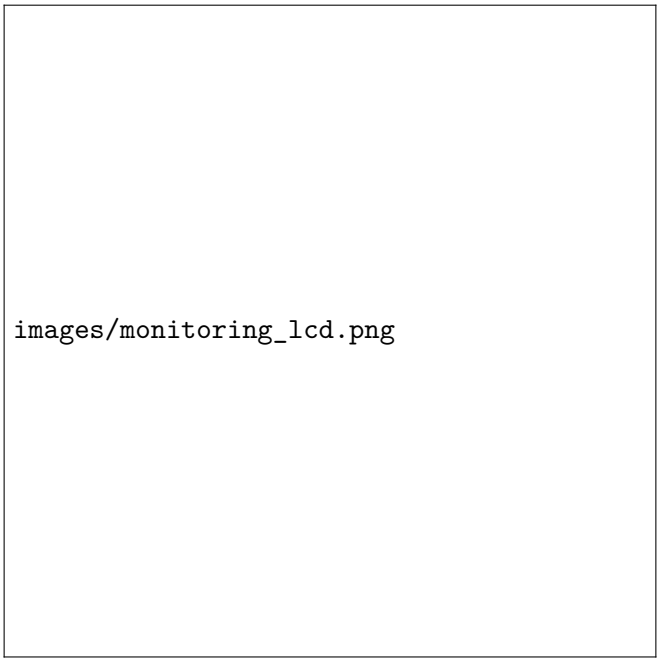


Figure 1: LCD displaying the latest temperature and humidity measurements.

4.2 Heater Behavior

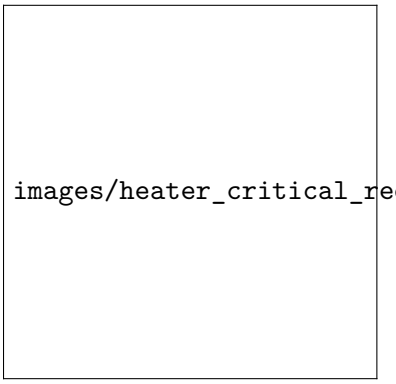
Figure **??** demonstrates the Heater indicator state transitions. The LED displays GREEN when the temperature is within the normal range (20–25°C), YELLOW when the temperature enters the warning range (15–20°C or 25–30°C), and RED when the temperature reaches a critical condition ($<$15°C or $>$30°C).

4.3 Cooler Behavior

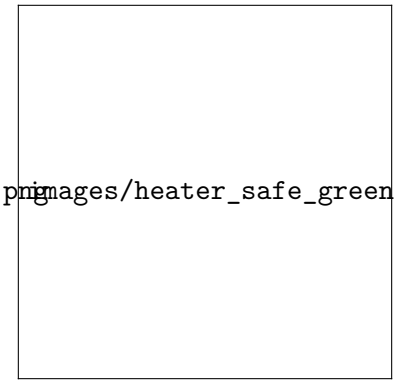
Figure **??** shows the Cooler Task activation. When the temperature exceeds 28°C, the cooler LED turns GREEN and remains active for 5 seconds. After the cooling interval expires, the LED turns OFF and the temperature is evaluated again.

4.4 Humidifier Behavior

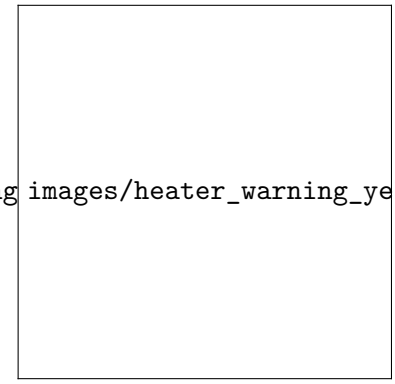
The Humidifier control sequence is described below. When humidity falls below 40%, the humidifier executes a timed sequence consisting of GREEN (5 s), YELLOW (3 s), and RED (2 s). After completion,



(a) Critical RED state.

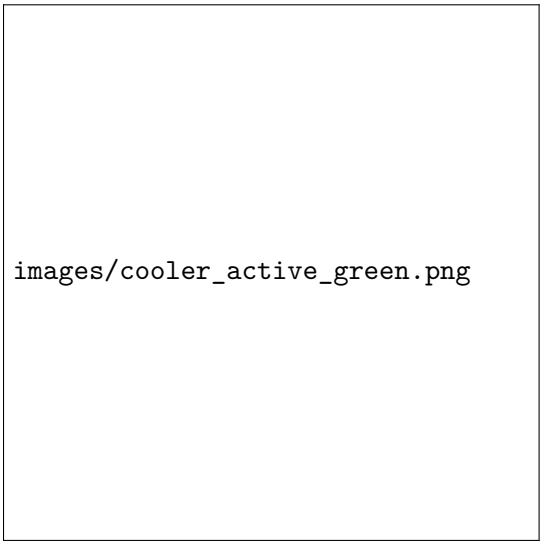


(b) Safe GREEN state.

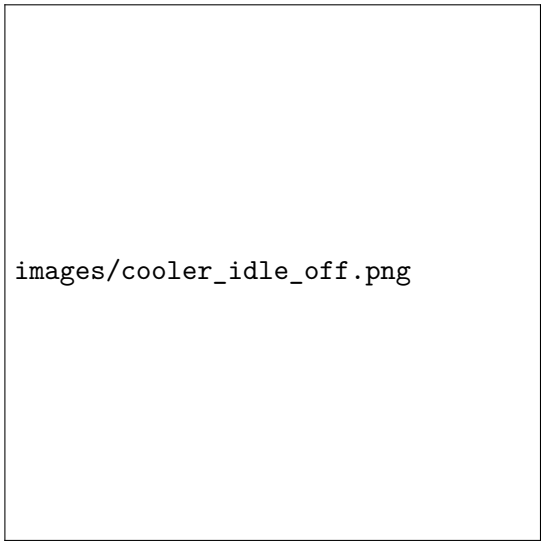


(c) Warning YELLOW state.

Figure 2: Heater indicator behavior for critical, safe, and warning temperature ranges.



(a) Cooler active in the GREEN state.



(b) Cooler inactive in the OFF state.

Figure 3: Cooler indicator during active cooling and after returning to the idle state.

the system returns to the idle state and rechecks the humidity condition.

4.5 Source Code Repository

GitHub link: https://github.com/Khainh2005/RTOSproject

5. Conclusion and Perspective

5.1 Summary of Achievements

The project successfully implemented an RTOS-based smart climate control system that monitors temperature and humidity and coordinates multiple actuator tasks using FreeRTOS-style asynchronous scheduling. The system fulfills the main course requirements, including periodic sensor acquisition, LCD display updates, heater state indication, timed cooler activation, and the multi-stage humidifier sequence. In addition, a dedicated Decision Maker task was introduced to coordinate actuator execution and improve synchronization between monitoring and control tasks. This design reduces the possibility of overlapping actuator activations and demonstrates the practical use of task coordination mechanisms in real-time embedded systems.

5.2 Limitations

Several simplifications were made in this project. First, the heater, cooler, and humidifier are represented only by RGB LED indicators rather than real physical actuators; therefore, the system does not actually modify the surrounding temperature or humidity. Second, the project was developed and validated primarily in a simulation environment, so additional testing on the YoluUNO hardware platform is necessary to evaluate timing accuracy, sensor reliability, and overall system behavior under real operating conditions. Furthermore, the current control logic is threshold-based and does not account for environmental dynamics such as delayed actuator effects or sensor noise.

5.3 Future Work

- Implement the system on physical hardware using real actuators such as fans, heaters, and humidifiers.
- Port the software from Python to C++ to achieve better integration with embedded RTOS environments and more efficient low-level hardware control.
- Measure the response time of the physical actuators and incorporate this information into the control logic to prevent unnecessary repeated activations.
- Extend the system with closed-loop control methods such as PID control for smoother and more accurate regulation of temperature and humidity.
- Add features such as wireless monitoring, data logging, fault detection, and power-saving operating modes.